

CAREERMITRA PROJECT DEPENDENCIES

This document lists the direct third-party packages, Python built-in modules, and local project modules used in this project. Transitive packages installed automatically by dependencies are not listed unless the project imports them directly.

THIRD-PARTY PACKAGES

Package: Used For

Flask: Flask is the main web framework used to create the web application, define routes, handle requests, manage sessions, and return responses. The project imports `Flask`, `abort`, `redirect`, `request`, `send\_file`, and `session`.

Werkzeug: Werkzeug utilities are used for Flask request handling support. The project uses `RequestEntityTooLarge` for upload-size errors and `secure\_filename` to safely clean uploaded file names.

Gunicorn: Gunicorn is the production WSGI server used by Render to run the Flask app. The Render start command is `gunicorn app:app`.

PyMuPDF: PyMuPDF is imported as `fitz` and is used to read uploaded PDF resumes. It helps extract text and process PDF content for skill analysis.

Matplotlib: Matplotlib is used to generate visual charts for resume analysis results. The project uses the non-GUI `Agg` backend so charts can be created on a server.

NumPy: NumPy is used for numerical operations while preparing analysis and chart data. It supports efficient calculations for resume skill matching outputs.

Pandas: Pandas is used for structured data handling and tabular processing. It helps organize analysis data before generating reports or charts.

PYTHON BUILT-IN MODULES

Module: Used For

`\_\_future\_\_.annotations`: This enables postponed evaluation of type annotations. It helps keep type hints cleaner and avoids some runtime annotation issues.

`html`: The `html` module is used to escape text before placing it into generated HTML. This helps prevent unsafe or broken HTML output.

`json`: The `json` module is used to read and write structured data. The project stores and loads records, metadata, analysis results, users, and contacts as JSON where needed.

`os`: The `os` module is used for environment variables such as host and port configuration. This lets the app run locally and on hosting platforms like Render.

`re`: The `re` module is used for regular expressions. It supports email validation, phone matching, skill matching, and text extraction patterns.

`uuid`: The `uuid` module is used to create unique IDs for users, records, resumes, job roles, and generated analysis results. These IDs help safely identify data across storage and database tables.

``zlib``: The ``zlib`` module is used for compression-related PDF text extraction helpers. It helps process compressed PDF stream content when extracting text manually.

``zipfile``: The ``zipfile`` module is used to inspect compressed document-style files when needed. It supports reading packaged file contents from zip-based formats.

``collections.Counter``: ``Counter`` is used to count repeated extracted words or skills. It helps summarize frequency-based data during resume analysis.

``datetime.datetime``: ``datetime`` is used to generate timestamps for users, records, resumes, messages, and analysis results. It also helps format dates for display.

``http.HTTPStatus``: ``HTTPStatus`` provides named HTTP status codes. The project uses it for clearer response handling, such as forbidden or no-content responses.

``pathlib.Path``: ``Path`` is used for safe, readable filesystem path handling. The project uses it to manage storage folders, templates, static files, uploads, and generated charts.

``typing.Any``: ``Any`` is used in type hints where values can have different shapes. It makes function signatures clearer without forcing overly strict types.

``urllib.parse.urlencode``: ``urlencode`` is used to build query strings for redirects. It helps pass messages and status types between pages.

``urllib.parse.quote``: ``quote`` is used to safely encode values inside URLs. The project uses it when building download links for stored files.

``xml.etree.ElementTree``: ``ElementTree`` is used to parse XML content. It supports extracting text from XML-based document data when needed.

``sqlite3``: ``sqlite3`` is used for the local application database. It stores users, profiles, job roles, resumes, analysis results, records, and contact messages.

``hashlib``: ``hashlib`` is used to hash passwords securely with PBKDF2. It helps avoid storing plain-text passwords.

``secrets``: ``secrets`` is used to generate secure random password salts. This improves password storage security.

``shutil``: ``shutil`` is used for filesystem operations such as deleting folders and stored resume data. It helps clean up uploaded files and generated results.

LOCAL PROJECT MODULES

Module: Used For

``app.py``: This is the main Flask application entry point. It creates the app, builds page contexts, handles resume analysis, and exposes ``app = create_app()`` for Gunicorn.

``frontend.py``: This module registers Flask routes for pages such as home, login, register, dashboard, upload, profile, result, contact, and delete actions. It connects web requests to handler functions in ``app.py``.

``backend.py``: This module contains backend helpers for template rendering and database connection setup. It centralizes shared low-level app support functions.

``data_store.py``: This module handles database and storage operations. It creates users, authenticates users, manages profiles, saves resumes, stores analysis results, deletes files, and loads dashboard data.

`db\_structure.py`: This module defines and initializes the SQLite database schema. It creates tables for users, profiles, job roles, skills, resumes, analysis results, records, and contact messages.

`skill\_library.py`: This module stores predefined job roles and required skills. The app uses it to compare uploaded resumes against selected target roles.

DEPLOYMENT PACKAGES FROM `REQUIREMENTS.TXT`

Flask>=3.0,<4.0

gunicorn>=21.2,<23.0

PyMuPDF>=1.24,<2.0

numpy>=1.26,<3.0

pandas>=2.2,<3.0

matplotlib>=3.8,<4.0